

Spam bot detection on Twitter platform using positional attention based dense convolutional neural network

Hemal Girishkumar Shah^{a,*}, Hiren Joshi^b

^a Department of Computer Science, Gujarat University, Ahmedabad, Gujarat 380009, India

^b Gujarat University, Ahmedabad, Gujarat 380009, India

HIGHLIGHTS

- To propose a new deep learning mechanism to efficiently detect and classify the twitter spam bots.
- For removing the irrelevant and redundant information pre-processing is performed using varied NLP methods.
- To extract essential features for mitigating the complexity issues, word embedding model and feature extractors are employed.
- To reduce the feature dimensionality problems, optimal features are selected utilizing Binary Water wheel plant.
- For classifying the spam bots and real users, Positional Attention based Dense Convolutional Neural Network is introduced.

ARTICLE INFO

Keywords:

Term Frequency-Inverse Document Frequency
Bidirectional Encoder Representations from Transformer
Binary Water wheel Plant Optimization
Positional Attention
Dense Convolutional Neural Network

ABSTRACT

Nowadays, social media platforms such as Twitter and Facebook play a major role in everyday life because of the incredible opportunities they offer users. Twitter is the most essential social media platform since it allows people to express themselves through tweets. However, because of its popularity among a vast number of users, the Twitter platform is being abused by automated accounts known as bots. Since automated accounts transmit fake news, fake ideas, and fake products, early detection of bots on the Twitter platform is critical. Previously, researchers introduced ineffective methods for identifying social media bots. Positional Attention-based Dense Convolutional Neural Network (PAtt_Dense CNN) is a new deep learning-based spam bot detection framework proposed in this paper. Pre-processing of the incoming data includes stop word removal, tokenization, stemming, n-gram identification, user mention, URL and hashtag removal, vocabulary density, and richness analysis. From the pre-processed images, significant features are extracted utilizing N-gram level vectorizer, TF-IDF vectorizer, character level vectorizer, and Extended Bidirectional Encoder Representations from Transformer (EBERT). After feature extraction, binary water wheel plant optimization is used to select the best characteristics. Finally, spam bot detection and classification are performed using a PAtt_Dense CNN. The performance of the proposed technique is then evaluated by analyzing the performance indicators and comparing them to existing procedures. According to the comparison results, the proposed technique achieved the enhanced outcome in terms of Accuracy, Precision, Recall, and F1-Score of 97.8%, 97.2%, 98.3%, and 99.2%, respectively.

1. Introduction

Online social networks (OSNs) have recently gained popularity among a variety of users. Because the users can share their opinions, news, organize events, etc. [1]. Twitter is one of the most significant social media platforms used by people of all ages. Tweets are used to communicate with both strangers and loved ones [2]. Users can also follow their favourite athletes, politicians, friends, and others. As a

result, users are kept up to date on relevant issues for discussion on a daily basis. However, automated accounts known as spam bots exploit the Twitter platform's services [3]. Tweets are posted by these bots with fake messages to attract the public. Bots spread rumours about politicians, certain products and services, and certain ideas. Thus, spam bots can mislead people into buying certain products that are of bad quality [4,5]. As a result, real users cannot differentiate between fake news and real news.

* Corresponding author.

E-mail addresses: hemalshah@gujaratuniversity.ac.in (H.G. Shah), hdjoshi@gujaratuniversity.ac.in (H. Joshi).

<https://doi.org/10.1016/j.asoc.2025.113725>

Received 12 August 2024; Received in revised form 16 June 2025; Accepted 4 August 2025

Available online 8 August 2025

1568-4946/© 2025 Elsevier B.V. All rights are reserved, including those for text and data mining, AI training, and similar technologies.

Bots are classified into four types: socialbots, spambots, cyborgs, and Sybils. Bots can operate within networks and perform specified actions on OSNs via control and command channels [6,7]. This automated account will use conventional services for nefarious operations, such as altering public opinion and promoting malicious products. Also, spam bots can cause wrong popularity, social phishing, spamming, cloning, or collision attacks. According to a recent study, the Twitter platform contains 9–15 % bot accounts, making it extremely vulnerable. As a result, bot identification on the Twitter platform is receiving a lot of attention because it serves as a defence mechanism against threats [8]. Because it is challenging to discern between actual users and spam bots, dynamic defences must be used. Botnets will emulate the interactions of Twitter users in order to avoid detection by bot detection techniques [9, 10]. Some of the bot-making tools include Use viral and Seek Socially. Some of the developed bots utilize cloud services such as Amazon Web Services or Heroku.

Spam bots generally allow for two types of operation: For example, they enable functions such as publishing, viewing, and replying to messages submitted by other users [11]. The second is networking skills, such as following other users or adding friends. Social media bots can do good deeds as well as malicious activities. Legal operations are controlled by verified profiles and genuine accounts owned by companies and celebrities [12]. Malicious bots will also harm users by doing malicious activities such as tracking user identities, canvassing voters to favour a specific politician, and spreading hate speech [13,14]. Therefore, there is a need to detect and classify spam bots on Twitter. Earlier researchers have introduced various techniques to detect spam bots on Twitter platforms [15–17]. Machine learning will be an effective method for detecting spam bots, and machine learning approaches include unsupervised, supervised, and reinforcement learning [30–32]. Commonly, machine learning techniques exhibit effective measures for detecting spam bots on the Twitter platform [33,34]. But they are more time-consuming and are expensive. Furthermore, they perform worse and learn more slowly for huge features [35–37]. As a result, some researchers have recently used deep learning algorithms to learn the features [18,19]. Deep learning is a subtype of machine learning that is defined by its layered architecture and ability to extract and process features from enormous volumes of data, such as text, images, and speech [20]. As a result, the proposed work introduces a new hybrid deep learning network to detect spam bots on the Twitter platform.

1.1. Motivation

Recently, the number of social media users has been increasing day by day, and people are unable to survive without using social media platforms. Twitter plays a significant role in social media platforms. People share their ideas, views, emotions, and suggestions through this platform. Still, automated accounts, known as bots, are indicated as unauthorized accounts that mislead users. Therefore, it is necessary to detect and classify the spam bots and real users. Previously, researchers presented many machine-learning algorithms for detecting spam bots. However, such strategies are ineffective since they employ a limited number of criteria for spam bot detection. Recently, researchers have also proposed certain deep-learning techniques for classification purposes. This technique deeply learns the features, allowing the system to achieve better classification performance. As a result, the proposed study uses a deep learning-based classification framework to identify and categorize spam bots. The following are the main goals of the planned work:

- To propose a new deep learning mechanism to detect and classify Twitter spam bots efficiently.
- To remove irrelevant and redundant information, pre-processing methods include stop word removal, tokenization, stemming, n-gram identification, user mentions, URL and hashtag removal, vocabulary density, and richness.

- To extract the significant features, use an N-gram level vectorizer, a Term frequency-inverse document frequency (TF-IDF) vectorizer, a character level vectorizer, and Extended Bidirectional Encoder Representations from Transformer (EBERT).
- To reduce the feature dimensionality and computational complexity, optimal features are selected utilizing Binary Water-Wheel Plant Optimization (BWPO).
- A Positional Attention-based Dense Convolutional Neural Network (PAtt_Dense CNN) is used to distinguish between spam bots and actual users.

The remainder of the paper is structured as follows: Section 2 discusses relevant works based on existing approaches, Section 3 describes the suggested methodology in depth, Section 4 provides a brief description of the result analysis, and Section 5 concludes and defines the feature scope.

2. Related works

Alhassun et al. [21] presented a Combined Text-Based and Metadata-Based Deep-Learning Framework for the Detection of Spam Accounts on Twitter. This paper identifies spam accounts on Twitter by generating an Arabic dataset appropriate for spam detection. Here, data labelling is performed by including flags for suspended accounts. An integrated architecture was introduced, depending on deep learning techniques, having various benefits, namely faster output, greater accuracy, and lower computational complexity. Text-dependent data with a convolutional neural network (CNN) architecture and metadata with a neural network model are used in this work. The outcome of the two architectures was classified as spam bots or non-spam bots. Still, the complexity of training two separate deep learning branches and effectively fusing their output was a challenging aspect.

Nabeela et al. [22] adopted a hybrid deep learning model that was Long short term memory (LSTM) or Bidirectional Encoder Representations from Transformers (BERT). There were various machine and deep learning-based approaches in use, which were difficult to spot on the internet. The news was sequential data; many RNN architectures were used, but they proved insufficient due to the diversity of the data. As a result, hybrid approaches to extracting content-based features using N-gram and TF-IDF were created. The spatial characteristics were retrieved using the LSTM or BERT model, and performance was assessed using two publicly available datasets. The designed model was effective in capturing the local and global features; still, the training time associated with the model was higher, and the over-fitting issues degraded the performance of the model.

Roshan et al. [23] presented an automated hybrid deep neural network for detecting and categorizing fake news. Approaches based on AI and natural language processing have garnered interest for automatically recognizing. Initially, input data are pre-processed through tokenization, term frequency-inverse document frequency transformation, and count vectorization. The detection is then accomplished by Convolution, Dynamic Semantic Structural Model (C-DSSM), and deep CNN (D-CNN), in which convolutional layers retrieve contextual information, max pooling evaluates feature vectors, and feature extraction is accomplished by C-DSSM, which extracts high semantic features and D-CNN for categorization. The LIAR dataset was discovered for the identification of fake news, and semantic matching in the model was considered one of the system's drawbacks.

Hayawi et al. [24] presented a hybrid deep neural network model for detecting social bots based on user profile data. A deep learning system is used in this article to categorize Twitter accounts as human or bot accounts. Tweeter accounts are classified based on features such as follower account, description, and tweet account. DeeProBot is a Deep Profile dependent bot detection system that was used in this case. The feature is supposed to be the text taken from the Twitter account. Then, the model is trained by raw text embedding utilizing already trained

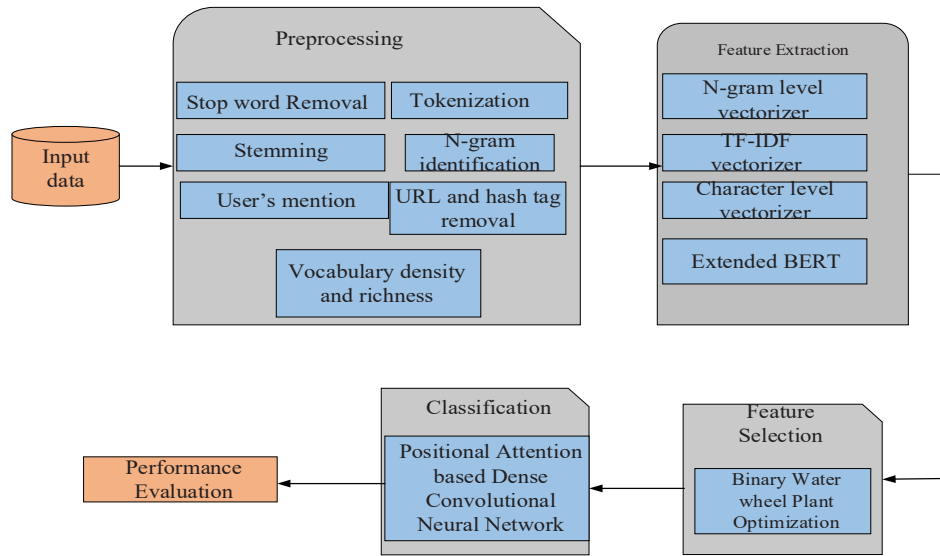


Fig. 1. Workflow of proposed study.

Global vectors (GLoVe) in order to represent the word. The DeepBotmodel accepts inputs such as mixed features involving binary, numerical, and text data. LSTM and thick layers are used to receive and process mixed input types. The designed technique was tested on a collection of publicly available labelled datasets. The designed model accomplished the enhanced outcome; still, the consideration of hand-crafted feature extraction limits the performance of the model.

Eiman et al. [25] presented a framework for stream-evolving bot detection (SEBD) that uses a PAC learning approach to maintain accuracy and confidence levels. Graph attention structures were used to construct the SEBD framework in this work. By utilizing fellowship links and profile elements, this structure is multi-faceted. After processing, it uses Apache Kafka to activate the Twitter API stream and determine the kind of account. A probably approximately correct (PAC) learning architecture for analyzing SEBD outcomes is presented. The framework's outputs are evaluated using cross-domain assessment techniques, including test holdout, benchmark data, machine learning classifiers, and a baseline tool. The designed model can identify and respond to concept drift in real time; however, the computational cost associated with the model is higher.

Ellaky et al. [26] introduced a hybrid DL method for detecting spam bots on social media platforms. In this study, automated accounts are said to be bots that disseminate wrong information, perform online propaganda, harass people, spread spam, etc. For the classification process, the input tweets are gathered from the Cresci-2017 and Twibot-20 datasets. The text representation of tweets is converted to vectors using the Glove method, followed by the classification of semantic text using a bi-directional gated recurrent unit (Bi-GRU) and LSTM model. However, the simulation results exhibited that the developed model was resistant to over fitting and effectively detected spam bots in unseen data.

Sallah et al. [27] have presented a social bot detection process using a new transformer based classification model. The traditional neural network based bot detection is more difficult to differentiate bots and human created text. So that, this study introduces a new transformer based architecture for detecting malicious tweets from the given data. Initially, text embedding is provided for making quality representation to improve the detection efficiency. Also, the classification process is done by using the transformer framework. In addition, explainable artificial intelligence (XAI) was utilized alongside transformer-based models for analyzing bots on social media and improves the transparency and reliability of developed models.

Vyas et al. [28] utilized ML techniques to extract efficient features to detect the malicious bots in twitter. The malicious bots create fake accounts, breach account information and spread wrong information. Hence exploratory study needed to identify the account features that helps to detect the malicious bots. The study adopted ML methods like information gain, correlation analysis and chi-square feature selection methods to detect top structures.

Problem Statement. : A lot of people use social media platforms, especially Twitter, which are a major concern in the modern world. People can share the messages simply by posting tweets. However, some of the automated accounts, called bots, misuse the Twitter platform by posting fake tweets. They introduce fake ideas, fake products, and fake news to mislead Twitter users. So, it is necessary to detect and classify spam bots as early as possible. In the past years, researchers have proposed some traditional techniques. However, such techniques are not effective; some of the existing techniques will not work well for a large number of features. Also, several machine-learning techniques are utilized for Twitter spam bot detection. However, machine learning techniques cannot produce superior results. So, there is a need for an effective technique to detect and classify spam bots more accurately.

3. Proposed methodology

Nowadays, social networks have become a crucial part of everyone's life, and as technology has advanced, so has people's ability to communicate via these platforms. Twitter is a popular social network that allows people to express themselves publicly. This has been used multiple times to create bot accounts that manipulate opinions and propagate bogus news. As a result, it is vital to distinguish between bot accounts and legitimate ones. Fig. 1 demonstrates the workflow diagram of the proposed technique.

On the Twitter platform, the suggested spam bot detection system consists of four major stages: pre-processing, feature extraction, feature selection, and classification. Tweets from various users are initially pre-processed to improve overall classification accuracy. The pre-processing stage involves several steps, including stop word removal, tokenization, stemming, n-gram identification, user mention, URL, and hashtag removal, as well as vocabulary density and richness. The pre-processed data is then sent to the feature extraction stage. Because the dataset contains sequential data and repeating words, the spam bot detection model's performance has been improved by extracting content-based features using an N-gram level vectorizer, TF-IDF vectorizer, Character

level vectorizer, and EBERT. The BWPO approach is used to pick the most optimal subset of features from the retrieved feature subset in order to solve the feature dimensionality issue. The feature vector is then provided to the next stage to detect the spam bot. Finally, spam bot detection and classification are carried out using the proposed PAtt_DenseCNN.

3.1. Pre-processing

Text pre-processing is the process of converting text into a reliable format for further learning and analysis. It assists in reducing unwanted data or noise by converting all characters to lowercase, removing stop words, and correcting punctuation errors, among other things. Pre-processing stages include stop word removal, tokenization, stemming, N-gram recognition, user mention, and determining word density and richness.

♦ **Stop Word Removal:** Stop words are words that are eliminated after or before the processing of text. The removal of articles and prepositions from text is known as stop words, and they are the most frequent words in text. It is important to find words that are meaningless by using linguistic knowledge and a large storage of documents. In linguistic knowledge, words with no meaning are found, and syllables are added to the words. The next approach finds the incidence of each term in the cache and then removes words that are often used. Eliminating these terms enhances the text processing's accuracy and precision. Examples of stop words include about, above, back, across, before, because, etc.

♦ **Tokenization:** Tokenization is the process of dividing long text sequences into tokens, which involves constructing a Bag of Words by removing question marks and punctuation. This requires a vast amount of data to be represented in an appropriate format. It avoids additional space and also breaks text apart to determine meaningful elements known as tokens; generally, word tokenization refers to the process of dividing sentences or posts into expressive tokens or words.

♦ **Stemming:** Stemming is the process of converting words to their original form. For example, sleep, sleeps, and sleeping might all be substituted with sleep. This is not appropriate for large datasets, as it will result in word differences because 'sleeping' is not the same as 'sleep.' This is only useful for tiny datasets.

♦ **N-grams identification:** Spammers utilize a set of misleading words in order to attract users. For the accurate detection of spam, n-gram identification is used. N-grams comprise a series of n words and, for instance, assume a phrase, 'this is a pencil'; here, it considers 'a' as one of the grams. But 'a' has no meaning, and the n-grams technique decides if this word is necessary or not. It considers only the words containing particular meanings.

♦ **Users Mention:** Users mention tagging, which is a way in which the person who creates the tweet communicates with a specific user and includes the specific user's user ID. Spammers have more domains than non-spam users, according to an analysis of existing datasets. Non-spam users identify a distinct domain for an individual, but spam users do not. Also, spam accounts will have different usernames and display names or a combination of the two.

♦ **Vocabulary density and richness:** The range of terms used in a dictionary is referred to as vocabulary richness, which indicates that if the text has more vocabulary, it is said to have vocabulary richness. Mean Word Frequency and Type Token Ratio (TTR) are two strategies for evaluating spam and non-spam vocabulary words. By examining spam as well as non-spam vocabulary in datasets, it is observed that spam tweets have less vocabulary, but non-spam tweets consist of many words. A spammer's purpose is to market a product and increase the number of followers; hence, they use a limited number of phrases. TTR examines the vocabulary in the text and determines whether these words are used in spam or non-spam messages.

The TTR is calculated by using the following Eq. (1)

$$TTR = \frac{\text{unique token in } E}{\text{tokens in } E} \quad (1)$$

Where, E specifies the dictionary. A token refers to the total amount of words used in a particular document. A large TTR score indicates high lexical richness, whereas a smaller TTR score indicates low lexical richness.

- Lexical density criterion (LD) is expressed as in the following Eq. (2)

$$LD = \frac{\text{words in } E \text{ excluding stop words}}{\text{token sin } E} \quad (2)$$

Where, E specifies the dictionary.

Stop words are not included in the LD computation since they have no semantic significance in text and are repeated in large numbers. Hence, the tweets are pre-processed utilizing natural language processing techniques (NLP). From the pre-processed data, certain significant features are extracted utilizing feature extraction techniques.

3.2. Feature extraction

After pre-processing, key characteristics are extracted from the pre-processed input to improve classification accuracy. For feature extraction, the suggested technique employs an N-gram level vectorizer, TF-IDF vectorizer, Character level vectorizer, and EBERT.

3.2.1. N-gram level vectorizer

The n-gram vectorizer is a feature extraction technique that uses unigram, trigram, and bigram as n-gram vectorizer models.

Unigram: The simplest language model format is unigram, which removes all conditioning contexts. Here, every term is determined independently, and this type of model is called a unigram language model. It is represented by the following equation (3):

$$Q(w_1, w_2, \dots, w_n) = Q(w_i) \quad (3)$$

Here, $w_1, w_2, \dots, w_n$ is a series of words and Q denotes the probability.

$$Q_{uni}(r_1 r_2 r_3 r_4) = Q(r_1)Q(r_2)Q(r_3)Q(r_4) \quad (4)$$

Here, r_1, r_2, r_3, r_4 denote a series of terms and Q represents the symbolic representation of probability.

Bigram: Bigram is a more complicated language model that is built on the terms of the unigram model and adds one word of context:

$$Q(w_i | w_1 w_2 \dots w_{m-1}) = Q(w_i | w_{z-1}) \quad (5)$$

Here, $w_1, w_2, \dots, w_m$ denotes the series of terms in the sentence and Q represents the probability function.

Trigram: Trigram is a more complex language model based on the bigram model. In this model, two words of context are added.

$$Q(w_i | w_1 w_2 \dots w_{m-1}) = Q(w_i | w_{z-2} w_{z-1}) \quad (6)$$

$$Q_{tri}(r_1 r_2 r_3 r_4) = Q(r_1)Q(r_3 | r_2 | r_1)Q(r_4 | r_3 | r_2)Q(r_5 | r_4 | r_3) \quad (7)$$

Here, r_1, r_2, r_3, r_4 denotes a series of terms.

3.2.2. TF-IDF

The product of term frequency and inverse document frequency is known as TF-IDF. A composite weight is produced for each term in all documents using TF-IDF. A phrase's "term frequency" (TF) is the number of times it appears in a single document.

$$TF(r) = \frac{\text{The amount of times a particular term appears in one document}}{\text{Total amount of terms in the document}} \quad (8)$$

Inverse Document Frequency (IDF): This IDF establishes the meaning of a specific word. All words are treated equally to calculate TF.

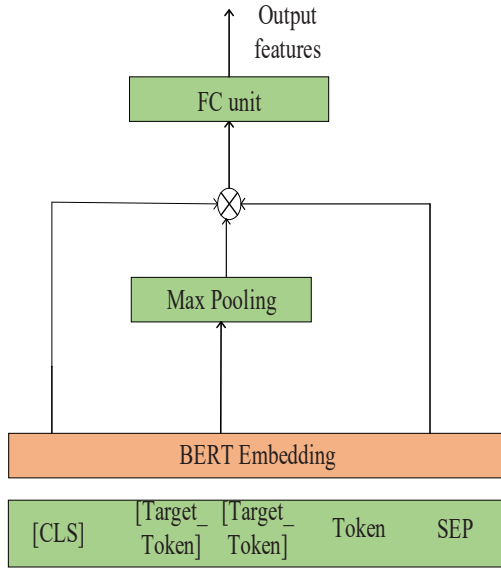


Fig. 2. EBERT model.

$IDF(r) = \log_e(\text{Total amount of documents} / \text{Amount of documents having word } r \text{ in it})$ (9)

$$TF - IDF = TF(r) * IDF(r) \quad (10)$$

3.2.3. Character level vectorizer

This is a character-level feature transformation technique that converts content information into a feature vector. Here, the input tweet content length is fixed with D characters. For this, if the length of the content is greater than D , then the data is truncated to the length D . If the length of the tweet content is less than D , then the data is repeated in the front part until the length of the data is equal to D . The character-level vectorization technique of interception of the preceding data, as well as filling, can help attain better results. For every entry, the sequence of characters is changed to the respective ASCII sequence value. If non-ASCII characters (characters that are not encoded in ASCII) are seen, such characters are dropped and, lastly, form the feature vector.

3.2.4. EBERT

Here, contextual embeddings of the text are extracted using the EBERT model. This BERT model is referred to as a bidirectional encoder, and it can read the full text at once. Fig. 2 illustrates the model of an EBERT. EBERT is efficient in capturing the finer linguistic cues indicative of spam or malicious intent, like unusual phrasing, excessive use of promotional language, or suspicious URLs embedded within the tweet's context. Also, by processing special tokens like Classification (CLS) and Separator (SEP), EBERT can learn representations for the entire sequence for capturing higher-level patterns associated with spam accounts. Besides, the EBERT is efficient in aggregating rich contextual features into a fixed-size representation of user behaviour that enhances spam classification. The aggregated feature representation by the EBERT representation encapsulates the learned user behaviour based on the textual content that makes the efficient feature extraction for training the spam detection model.

The encoder architecture known as BERT tokenizes the input sequences into individual words. Next, the vectors from the location, token, and segment embedding layers are integrated to produce a vector with a fixed length and dimension of 128. In order to distinguish between the start and finish of a sentence, a special token called CLS and SEP has been implemented. CLS serves as a token for classification tasks, and the transformer's encoder unit receives the final word embedding vector. There are twelve encoder layers in the BERT-base design. An attention matrix is calculated for each word vector, and then all of the

attention matrices are added together. The Feed-forward Network (FFN) receives the output vector in parallel following the eight multi-head attention blocks. Subsequently, the encoder in use forwards the result of this FFN to the subsequent encoder. BERT has twelve encoders, so this operation is executed twelve times. Finally, the transformer's encoder unit produces a context-dependent word embedding vector with dimensions 768. The vector is then provided to two fully linked units with dimensions of 768 and 32 and finally to a dropout unit with a probability ratio of 0.2. Cross-entropy is a loss function that serves as the objective function during the BERT training phase. Commonly, the output produced at the CLS tag is provided as input to the fully connected unit. In the proposed work, the max-pooling of the target terms is integrated with the output at CLS. This integrated output is fed into the fully linked unit as an input. The important features are chosen and sent to the feature selection step using enhanced BERT.

3.3. Feature selection using BWPO algorithm

After feature extraction, optimal features are selected using Binary Water Wheel Plant Optimization. Water Wheel plants follow a particular strategy to catch their prey. This strategy is used in the proposed work to select optimal features. In this case, the water wheel plant serves as the search agent, while the prey serves as the feature. The mathematical model of the BWPO is explained in the following equation.

Initialization: BWPO is a strategy that utilizes a set of search agents to achieve a satisfactory outcome for an issue through numerous efforts in the search region. A matrix is used to describe the complete BWPO algorithm. During the initial phase of BWPO, the search agent's starting places are found at random in the search area.

$$Q = \begin{bmatrix} Q_1 \\ \vdots \\ Q_i \\ \vdots \\ Q_M \end{bmatrix} = \begin{bmatrix} q_{1,1} & \cdots & q_{1,j} & \cdots & q_{1,n} \\ \vdots & \ddots & \vdots & \ddots & \vdots \\ q_{i,1} & \cdots & q_{i,j} & \cdots & q_{i,n} \\ \vdots & \ddots & \vdots & \ddots & \vdots \\ q_{M,1} & \cdots & q_{M,j} & \cdots & q_{M,n} \end{bmatrix} \quad (11)$$

$$Q_{ij} = LB_j + s_{ij}(UB_j - LB_j), i = 1, 2, \dots, M, j = 1, 2, \dots, n \quad (12)$$

Here, M and n specifies the amount of search agents and the amount of variables correspondingly; s_{ij} represents a random number in the range $[0,1]$, UB_j and LB_j specifies the upper bound and lower bound present in the j^{th} problem variable. Q specifies the population matrix having search agent locations, Q_i denotes the i^{th} search agent, and q_{ij} is the problem variable. The fitness function of the BWPO is given in the following Eq. (13).

$$g = \lambda * (1 - cl(A[: 1^{ind_i}])) + \left((1 - \lambda) \frac{|G|}{E} \right) \quad (13)$$

Here, λ denotes the control parameter, $A[: 1^{ind_i}]$ represents the tuning parameter, and 1^{ind_i} specifies the total amount of 1's present in the array. Each search agent represents a distinct method of solving the problem, and each can have its fitness evaluated separately. Eq. (14) demonstrates how the goal function can be simply expressed as a vector.

$$G = \begin{bmatrix} G_1 \\ \vdots \\ G_i \\ \vdots \\ G_M \end{bmatrix} = \begin{bmatrix} G(A_1) \\ \vdots \\ G(A_i) \\ \vdots \\ G(A_M) \end{bmatrix} \quad (14)$$

Here, G represents a vector having the entire objective function values and G_i is the value predicted for i^{th} search agent. This objective function is utilized to obtain the best solution. The best solution is represented by a greater value of the objective function. Because of the random movement of the search agents along the search zone, the current optimal solution will change with each repetition.

Phase 1: Exploration Phase

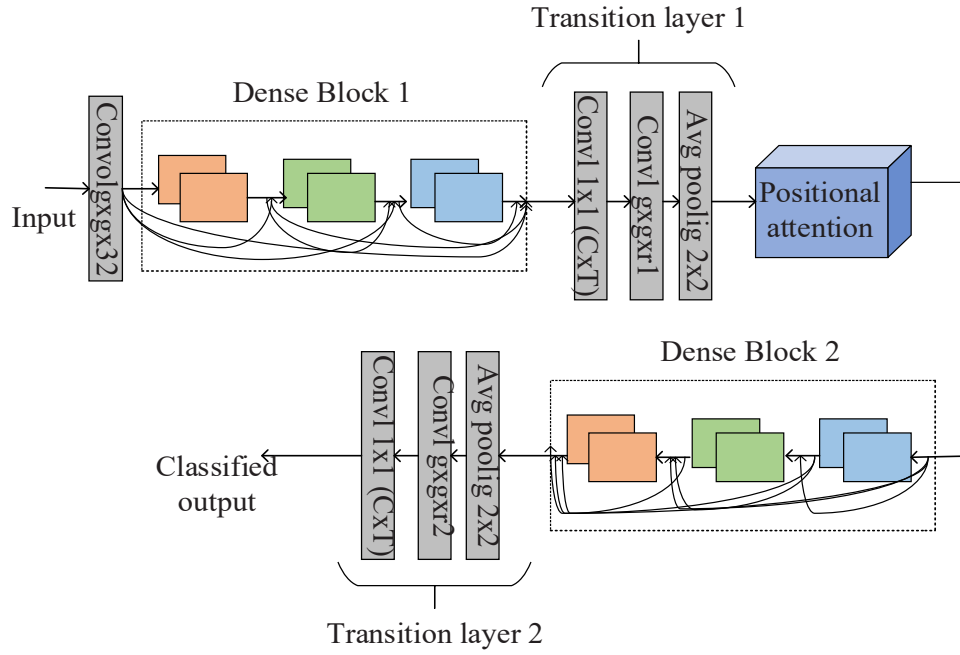


Fig. 3. Architecture of PAtt_DenseCNN model.

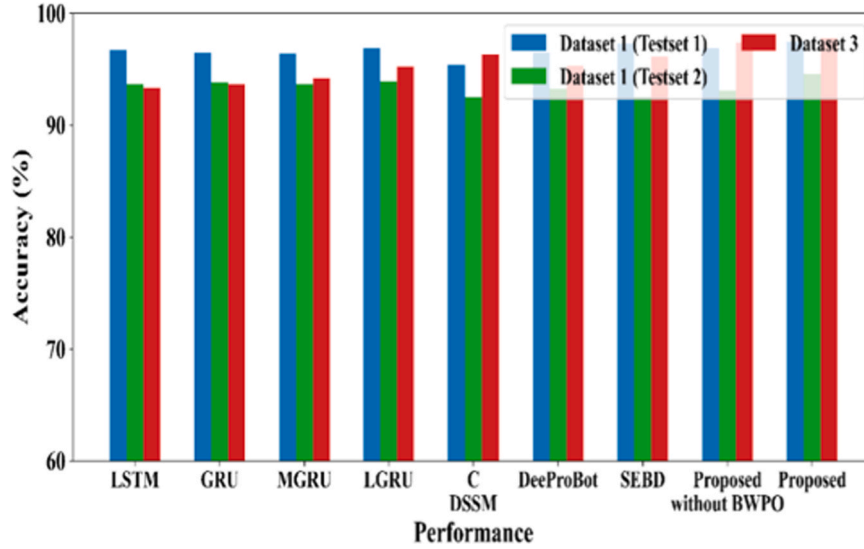


Fig. 4. Accuracy performance analysis.

A search agent will start looking for the feature, and its position in the search space may change. By replicating the search agent's attack on the feature, large variations in the position of the search agent are generated. BWPO's exploring ability is improved by discovering the best spot and escaping from local optima. Here, Eq. (15) is utilized to attain the new location of the search agent.

$$V = s_1 \cdot (Q(r) + 2L) \quad (15)$$

$$Q(r+1) = Q(r) + V \cdot (2L + s_2) \quad (16)$$

Here, s_1 and s_2 represents random variables containing values among [0,2] and [0,1], V represents a vector where the search agent searches for potential locations, and L denotes an exponential variable having values between the range [0,1]. The search agent modifies its location and provides the solution, as given in Eq. (17), if it remains unchanged after three iterations.

$$Q(r+1) = \text{Gaussian}(\eta_Q, \sigma) + s_1 \left(\frac{Q(r) + 2L}{V} \right) \quad (17)$$

Phase 2: Exploitation Phase

In the exploitation phase, when the search agent tries to capture the feature, a small location change occurs. This enhances the exploitation capability of local search, resulting in optimal solutions that occur closer to previously discovered solutions. When the value of the target function is more significant than the actual position, the search agent is found in a new place, as shown by the following Eq. (18).

$$V = s_3 \cdot (LQ_{best}(r) + s_3 Q(r)) \quad (18)$$

$$Q(r+1) = Q(r) + Q(r) + LV \quad (19)$$

Here, r_3 specifies a random variable having values in the interval [0,2], $Q(r)$ represents the present solution at iteration r and Q_{best} denotes

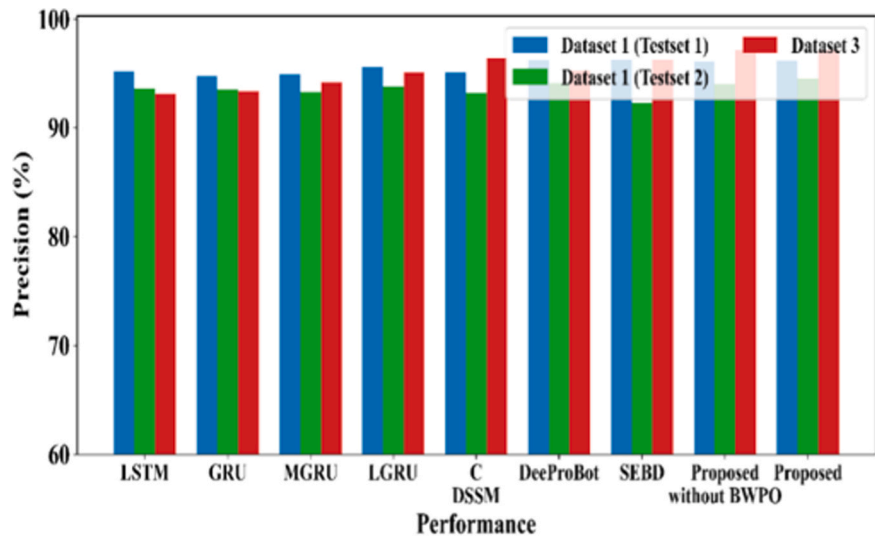


Fig. 5. Precision performance analysis.

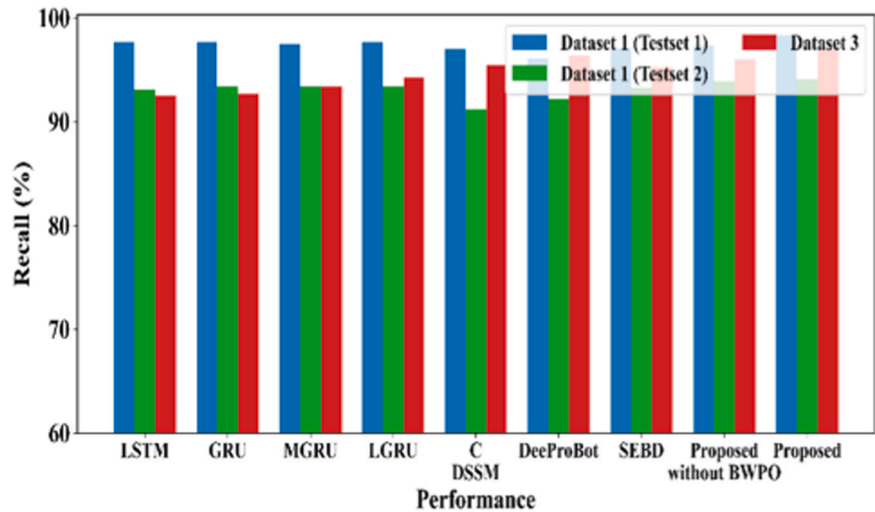


Fig. 6. Recall performance analysis.

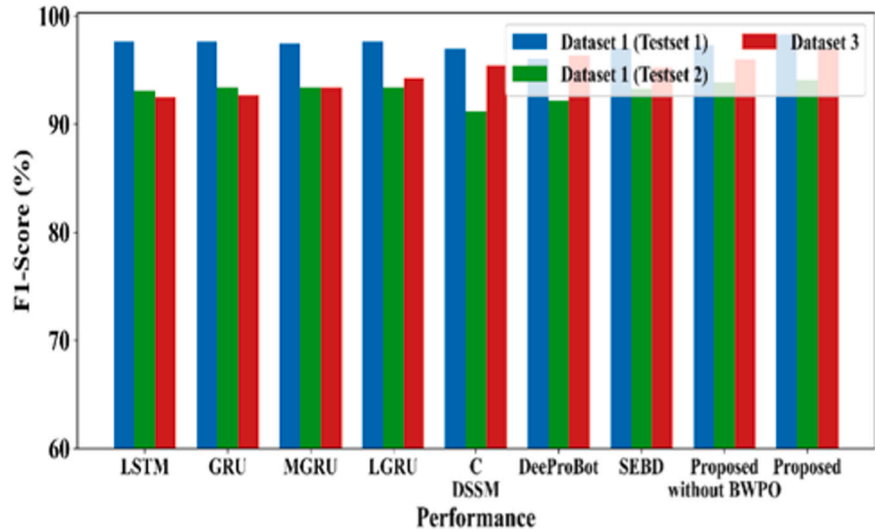


Fig. 7. F1-score comparison analysis.

the best solution. Similar to exploration, a mutation is used to help break free from local minima if the solution is not improved after three iterations.

$$Q(s+1) = (s_1 + L) \sin\left(\frac{G}{D} \theta\right) \quad (20)$$

Here, Q and D denotes the random variables having values in the interval $[-5, 5]$. Also, the value of L degrades exponentially utilizing Eq. (21).

$$L = \left(1 + \frac{2 * r^2}{R_{\max}} + G\right) \quad (21)$$

Binarization of search space: Search agents whose representations provide a binary search space will comprise the BWPO search space. All of the people in the search space are represented by binary numbers. Initially, the number of search agents in the search area is fixed by the size of the population, pop size, and the dimension of the dataset E . Here,

the $BWPO \oplus$ function gives the optimal solution in the interval 0–1 along every element E in ind_i . Every ind_i would receive results if the entire optimization process is completed. Cells having values of 1 s represent selected features. To find the best solution, compute the fitness value. Then, use the following sigmoid function to transform the continuous BWPO solution to binary.

$$Binary = \begin{cases} 1 & \text{if } sigmoid(R_{best}) \geq 0.5 \\ 0 & \text{otherwise} \end{cases} \quad (22)$$

$$Sigmoid(R_{best}) = \frac{1}{1 + e^{-10(R_{best} - 0.5)}} \quad (23)$$

Thus, the solutions are optimized in a discrete solution space. Thus, optimal features were selected utilizing this BWPO. The selected features are then fed to the classification stage for classification. The pseudocode of the utilized BWPO algorithm is given in Algorithm 1.

Algorithm 1. Pseudocode of BWPO algorithm

```

Initialize the position of the search agents  $Q_i (i = 1, 2, \dots, m)$  for  $m$  search agents,
fitness function  $g_m$  iterations  $r$ ,  $r_{\max}$ , and parameters of  $s, s_1, s_2, s_3, g, d$ , and  $L$ 
Binarize the solution area
Compute the fitness  $g_m$  for every location  $Q_i$ 
Find an ideal search agent position  $Q_{best}$ 
Fix  $r = 1$ 
While  $r \leq R_{\max}$  do
  for  $(i = 1 : i < m + 1)$  do
    if  $(s < 0.5)$  then
      Explore the search area utilizing:
       $V = s_1 \cdot (Q(r) + 2L)$ 
      If the solution will not change for three iterations
      then
         $Q(r+1) = \text{Gaussian}(\eta, q, \sigma) + s_1 \left( \frac{Q(r) + 2L}{V} \right)$ 
      End if
    Else
      Exploit the present solutions for getting the ideal solution utilizing
      equations (16) and (19)
      If the solution will not change for three iterations
      then
         $Q(r+1) = (s_1 + L) \sin\left(\frac{G}{D} \theta\right)$ 
      End if
    End if
  End for
  Decrease the score of  $L$  exponentially utilizing:
   $L = \left(1 + \frac{2 * r^2}{(R_{\max})^3} + g\right)$ 
  Update  $s, s_1, s_2, s_3, g, d$ 
  Compute the objective function  $g_m$  for every location  $Q_i$ 
  Find the ideal location  $Q_{best}$ 
  Fix  $r = r + 1$ 
End while
Change the ideal solution to a binary
Return the ideal solution.
  
```

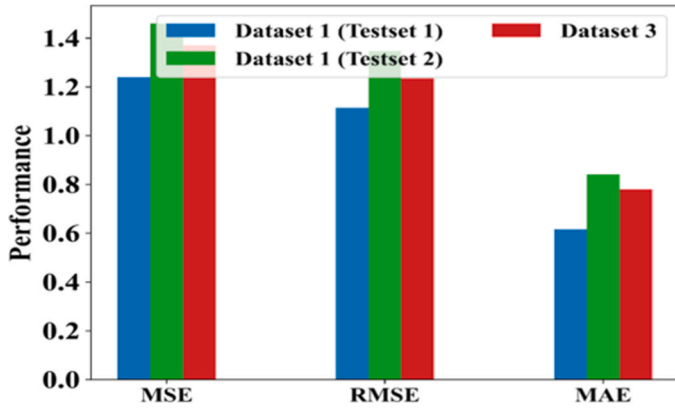


Fig. 8. MSE, RMSE, and MAE analysis of the proposed model.

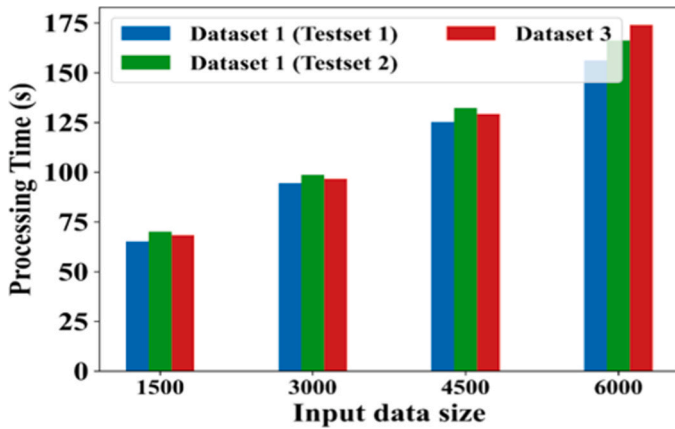


Fig. 9. Processing time (seconds) of the proposed technique.

3.4. Classification using PAtt_DenseCNN model

The selected features are then fed to the classification stage for classifying tweets as spam bots and real users. The proposed work uses positional attention-based dense convolutional neural networks for classification. The structure of PAtt_DenseCNN is illustrated in Fig. 3.

There are two dense units in the densely connected convolutional neural network, in which every dense unit consists of $k = 4$ convolution layers. In a dense unit, every convolution layer generates $S = 24$ amount of feature maps, having a filter size $g = 3$. The output of every dense unit constitutes the input of the next transition layer. Every transition layer is constructed of a bottleneck layer where $D = 4$ and $S = 24$, followed by a convolution layer having a filter size of $g = 3$, and finally, a pooling unit.

Dense unit: DenseNet mainly focuses on feature reuse by using dense connectivity to enhance the efficiency of the parameters. Also, the dense block enhances feature propagation as well as gradient flow. Here, the given dense layer, having k convolution layers, is integrated and given as input to the next convolution layers in a dense unit. This extensive connectivity, which connects every preceding layer to the following layer, improves information flow between convolution layers and enables good feature propagation and gradient flow between convolution layers. So, the k^{th} convolution layer consists of integrated feature maps produced by the convolution layers that follow it in a given dense unit, represented as $A_0, \dots, A_{k-1}$ as input:

$$A_k = G_k([A_0, A_1, \dots, A_{k-1}]) \quad (24)$$

Here, the integrated feature maps are represented by $[A_0, A_1, \dots, A_{k-1}]$, which are produced by the next convolution units from $0, 1, \dots, k - 1$.

$G_k(\cdot)$ is represented as a composite function that performs the operations, namely rectified linear unit (ReLU), batch normalization (BN), and convolution (Conv). Also, every $G_k(\cdot)$ generates T feature maps and T is regarded as the growth rate of DenseNet. The growth rate is the total number of feature maps that a given convolution layer in a dense unit will produce. Then, k^{th} convolution layer consists of $T_0 + T \times (k - 1)$ number of feature maps as input. Here, T_0 represents the number of feature maps for the input layer in a dense unit. Consider s as the number of feature maps that are given as input to the dense unit containing k convolution layers. Here, i denotes the dense block, and the number of feature maps output by the dense unit is represented by s_i and can be computed as in the following Eq. (23).

$$s_i = s + (k \times T) \quad (25)$$

Transition layer: The transition layer in the dense unit consists of $1 \times 1 \times s_i$ convolutional layer, in which s_i denotes the number of output feature maps produced by the dense unit. This is followed by a pooling layer and also a 1×1 bottleneck layer, which is placed before the $g \times g$ convolution layer and the pooling layer. 1×1 convolution layer is utilized for the best modelling of the local. By lowering the quantity of input feature maps, the bottleneck layer lowers computational complexity and improves operational efficiency. By using this bottleneck layer, features propagate more effectively in a bottleneck manner, and superfluous features are filtered out by the convolution layers. Here, g represents the convolution layer's filter size, and C represents a constant for the bottleneck layer. Also, S_i amount of feature maps are provided as input; the transition layer utilizes $1 \times 1 \times (C \times T)$ convolution as the bottleneck layer, and $g \times g \times s_i$ convolution is given as the convolution layer and the pooling layers. In addition, the convolution function in the transition layer is ReLU and the batch normalization layer.

Every transition layer generates feature maps that are made up of different features. A collection of integrated feature maps created by every convolution layer inside a dense unit is the dense unit's output. This uses $1 \times 1 \times (C \times T)$ bottleneck unit to pass the various features along the feature maps within a bottleneck and thus again improves the propagation of features in the network. The undesirable features are then removed by the transition layer's convolution layer. Ultimately, a categorized output is produced that helps to distinguish between actual users and the spam bot. Besides, the utilization of max pooling, other than average pooling, further enhanced the network's ability.

Attention layer: The suggested approach uses an attention layer to lessen bias and noise; a positional attention layer is employed to gather the most crucial portions of the tweets and is provided by

$$B_i = \sum_{l=1}^{|W|} \beta_l L_i \quad (26)$$

Here, β_l can be expressed as

$$\beta_l = \frac{\exp(j^x \tanh(V^x[L_i \oplus q_i]) + y^x)}{\sum_{l=1}^{|W|} \exp(j^x \tanh(V^x[L_l \oplus q_l]) + b^A)} \quad (27)$$

Here, $V^x \in S^{(e_h + e_p) \times (e_h + e_p)}$, $y^x \in S^{e_h \times e_p}$ and $j^x \in S^{e_h \times e_p}$ are matrices which are learnable, the concatenation function is denoted as $\oplus$. $q_i \in S^{e_p}$ is the position weight of the i^{th} word and computed as follows:

$$q_i = \begin{cases} \frac{|i - begin|}{n} & i \leq start \\ 0 & start \leq i \leq end \\ \frac{|i - end|}{n} & i > end \end{cases} \quad (28)$$

Here, *begin* and *end* are the starting and ending order of the tweet, correspondingly.

Finally, a categorized output that distinguishes between the spam

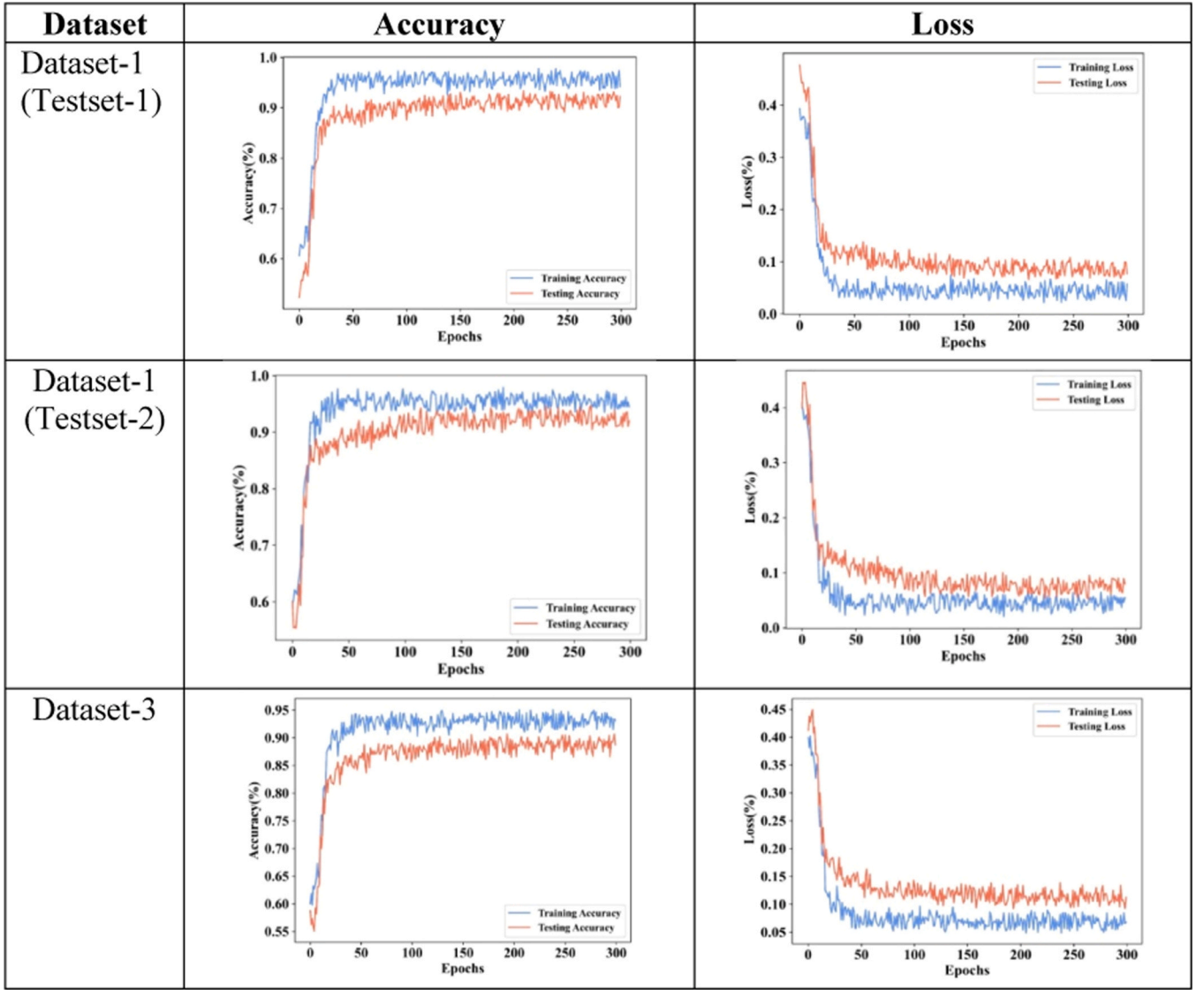


Fig. 10. Accuracy and loss curves.

bot and genuine users is obtained. In addition, the use of max pooling rather than average pooling in the transition layer improved network capability.

4. Results and discussion

The experimental results of the proposed PAtt_DenseCNN technique are described in depth in this section. Cresci 2017 [29] is the dataset under consideration here, and it is separated into two test sets. Test set 1 (Dataset-1) contains 1982 accounts and 4061598 tweets, while the test set 2 (Dataset-2) contains 928 accounts and 2628181 tweets. The following is a link to the dataset:

<https://botometer.osome.iu.edu/bot-repository/datasets.html>

The proposed technique's findings are compared to those of four other techniques: LSTM, Gated Recurrent Unit (GRU), Minimal Gated Unit (MGU), and Lightweight Gated Recurrent Unit (LGRU). The goal of this comparison is to show the effectiveness of the proposed technique. The findings are evaluated using performance criteria such as accuracy, precision, recall, and F1 score.

Dataset 3: The dataset is taken from the Kaggle repository https://www.kaggle.com/datasets/goyaladi/twitter-bot-detection-dataset?select=bot_detection_data.csv with 12 various attributes for each data.

4.1. Performance metrics

The performance metrics of the proposed technique show improved performance. The performance metrics utilized for comparing the proposed technique are explained as follows:

Accuracy: Accuracy is a measure of how near the predicted value is to the reference value. It can be mathematically expressed as in the equation below.

$$Accuracy = \frac{TP + PN}{TP + TN + FP + FN} \quad (29)$$

here, TN denotes the True Negatives, which are correctly classified as spam bots, FP refers to the False Positives, TP denotes the True Positives, and FN provides the False Negatives.

Precision: The ratio of expected true positive values to the sum of both true positive and false positive values is referred to as precision. It is provided by

$$Precision = \frac{TP}{TP + FP} \quad (30)$$

Recall: The ratio of observed true positive findings to the total anticipated true positive and false negative values is known as recall,

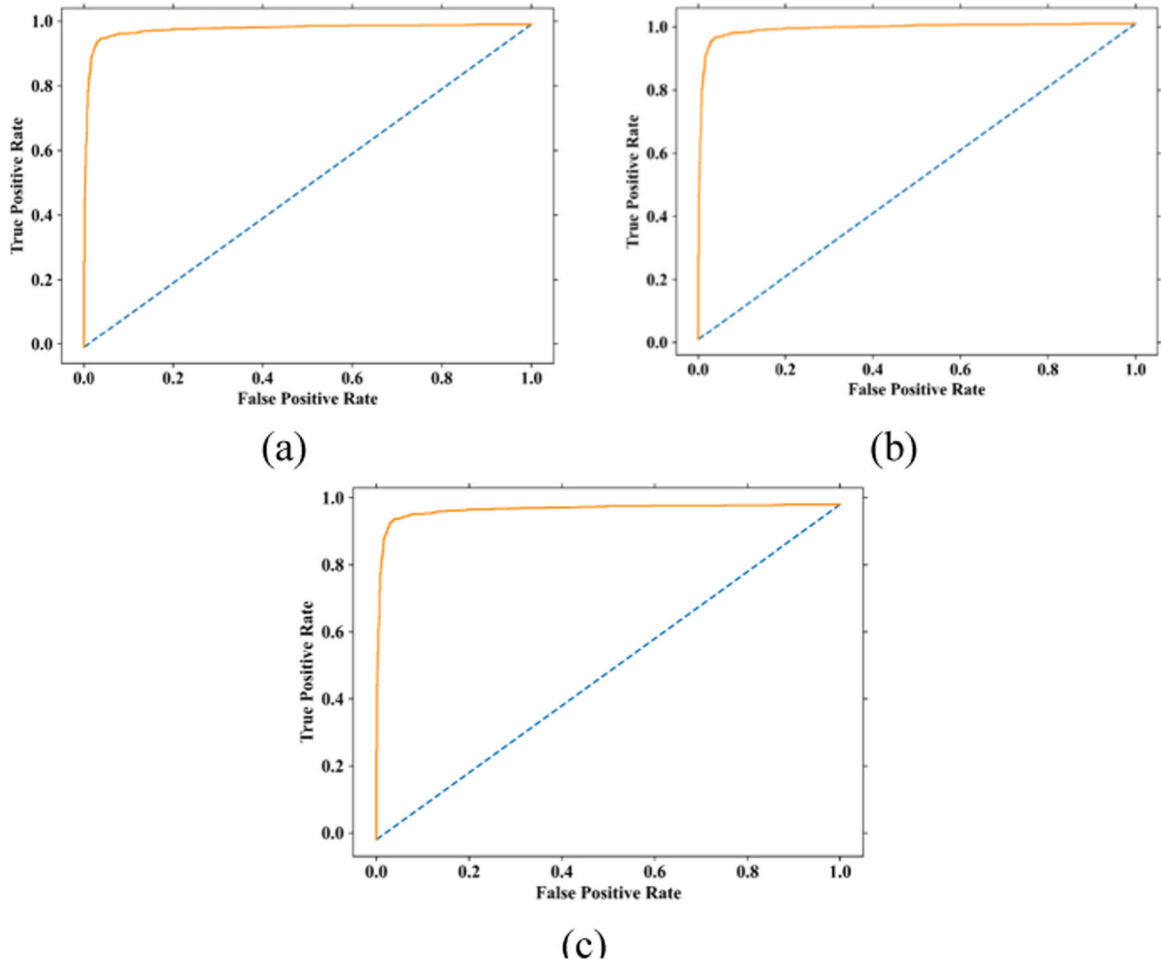


Fig. 11. RoC curve of the proposed technique (a) Dataset 1 (Testset-1), (b) Dataset 1 (Testset-2), and (c) Dataset 3.

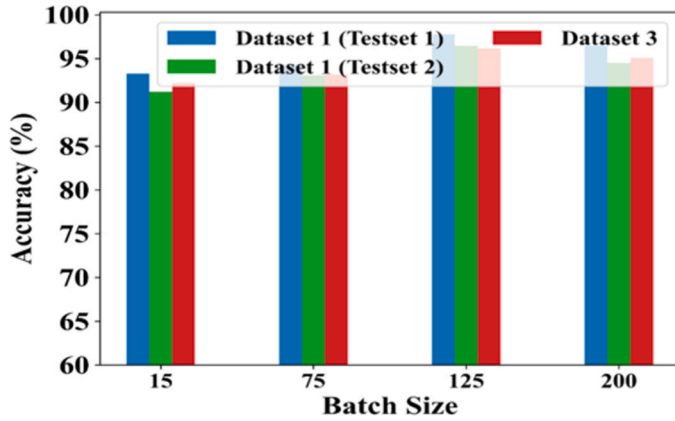


Fig. 12. Accuracy with reference to Batch size.

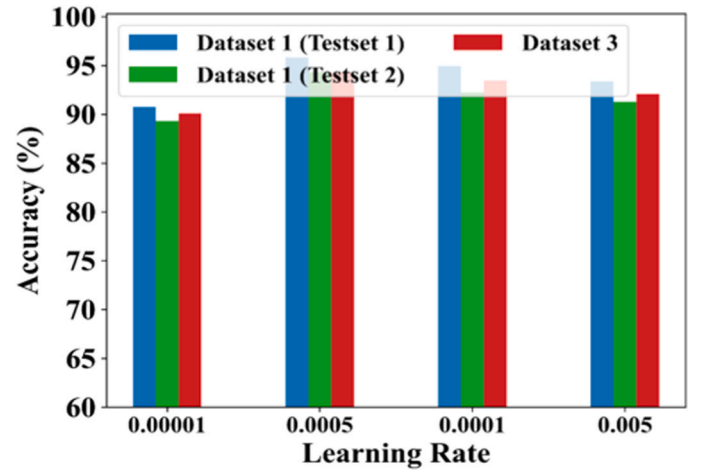


Fig. 13. Accuracy with reference to learning rate.

which is also known as sensitivity. It is given by

$$\text{Recall} = \frac{TP}{TP + FN} \quad (31)$$

F1 score

F1Score is regarded as a variant of accuracy, which will not be affected by the negatives. It is illustrated in the equation below and can be mathematically represented as in the following equation.

$$2 \frac{\text{Recall} \times \text{Precision}}{\text{Recall} + \text{Precision}} \quad (32)$$

Mean Square Error (MSE): The mean squared difference (MSE) between the observed and expected values is an often used metric. If no error occurs, the value of MSE is Zero. It is expressed in the following Eq. (33).

$$MSE = \frac{1}{t} \sum_{i=1}^t (G_i - \hat{G}_i)^2 \quad (33)$$

Here, G_i represents the observed values, $\hat{G}_i$ denotes the predicted

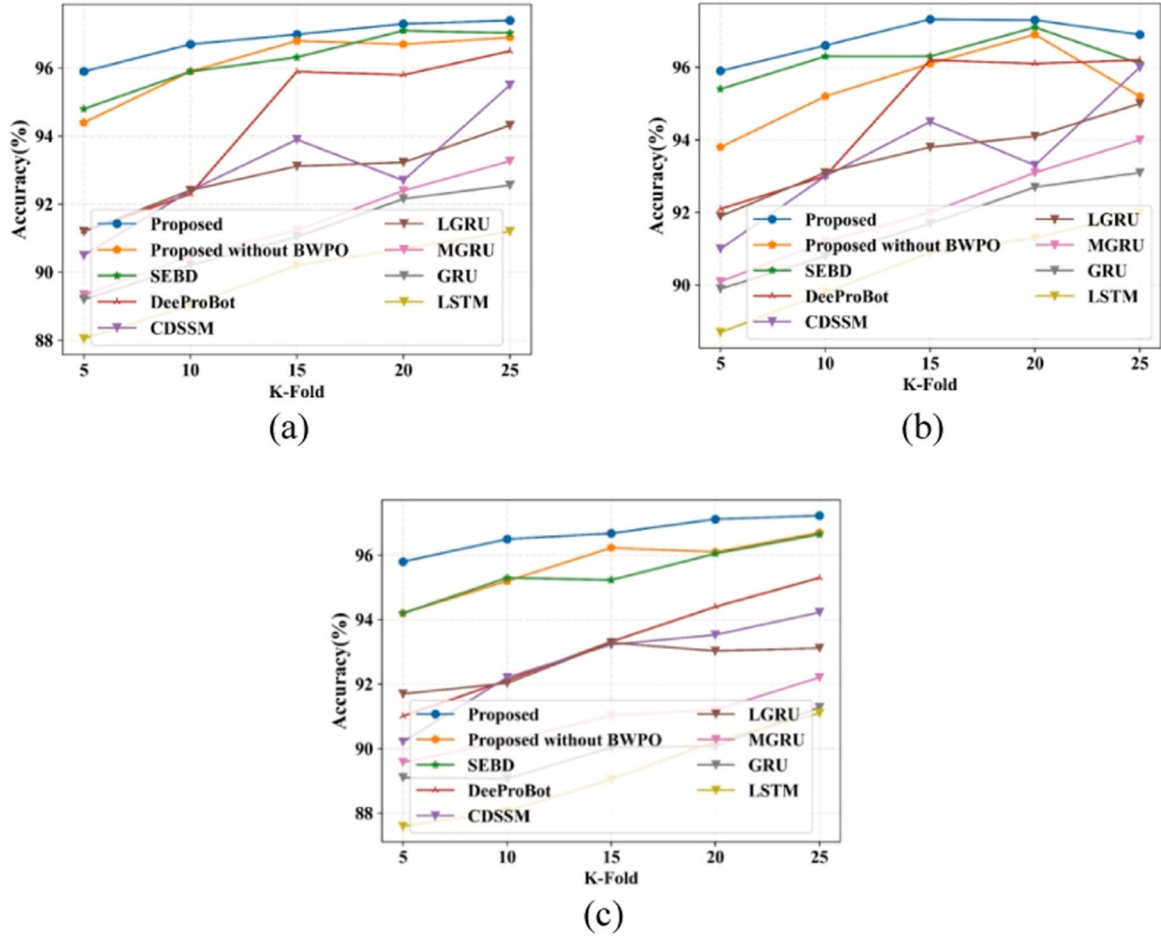


Fig. 14. Analysis based on K-Fold validation: (a) Dataset 1 (Testset-1), (b) Dataset 1 (Testset-2), and (c) Dataset 3.

values, and t specifies the number of observations.

Root Mean Square Error (RMSE): The square root of the mean squared difference between the observed and anticipated values is known as the root mean square error, or RMSE. It is expressed in the following Eq. (34).

$$RMSE = \sqrt{\frac{1}{t} \sum_{i=1}^t (G_i - \hat{G}_i)^2} \quad (34)$$

4.2. Performance analysis

The performance of the suggested technique is evaluated in this part by comparing it to current procedures. By viewing the comparison graphs, it can be understood that the proposed techniques are far better than the existing techniques.

Fig. 4 compares the proposed technique for set 1 with known techniques, including LSTM, GRU, MGU, and LGRU, in terms of accuracy. The graph shows that the proposed technique achieves a higher accuracy, with a rate of 97.5 %. Here, LSTM achieves an accuracy of 96.7 %, GRU achieves 96.55 % accuracy, MGU achieves 96.4 % accuracy, and LGRU has an accuracy of 96.9 %. Thus, the existing techniques show lower accuracy values, thereby giving poor performance. The proposed system's enhanced accuracy value demonstrates the usefulness of the proposed classification technique in classifying spam bots and real users. The proposed model achieves high accuracy due to robust feature extraction techniques, such as TF-IDF, character-level vectorization, and BERT, which preserve semantic and sequential features in tweets. The PAtt_Dense CNN technique further enhances the accuracy by learning contextual relationships in the data. The BWPO feature selection reduces

noise and redundancy, which results in more accurate predictions for the proposed model.

Fig. 5 demonstrates the comparison of precision for the proposed technique when set 1 is considered with existing techniques, namely LSTM, GRU, MGU, and LGRU. The graph clearly shows that the proposed technique has a high precision score, with a precision of 96.2 %. Here, the LSTM has a precision of 95.2 %, the GRU achieves 94.8 % precision, the MGU achieves a 94.9 % precision value, and the LGRU has a precision of 95.6 %. Thus, the existing techniques reveal lower precision values, thereby giving poor performance. The proposed system's greater precision value demonstrates the efficiency of the proposed classification technique for distinguishing between spam bots and legitimate users. Here, the integration of n-gram and semantic feature representation enables the model to learn detailed patterns that are highly useful in detecting spam bots, resulting in an improved precision rate of the proposed model.

Fig. 6 shows the comparison of recall for the proposed technique with four existing techniques, namely LSTM, GRU, MGU, and LGRU if set 1 of the input tweets is considered. From the graph, it is observed that the proposed technique has a good recall score, achieving a recall of 98.3 %. The recall values for the existing approaches are as follows: LSTM (97.7 %), GRU (97.7 %), MGU (97.5 %), and LGRU (97.7 %). Here, it is clearly seen that the existing techniques show minimum recall values, thereby causing degraded performance. The proposed system's higher recall value demonstrates the efficacy of the proposed classification technique in distinguishing between spam bots and real users. Here, the consideration of EBERT in feature extraction with a deep contextual understanding of tweet content assists the proposed model in enhancing recall. In addition, EBERT detects finer expressions and bot-

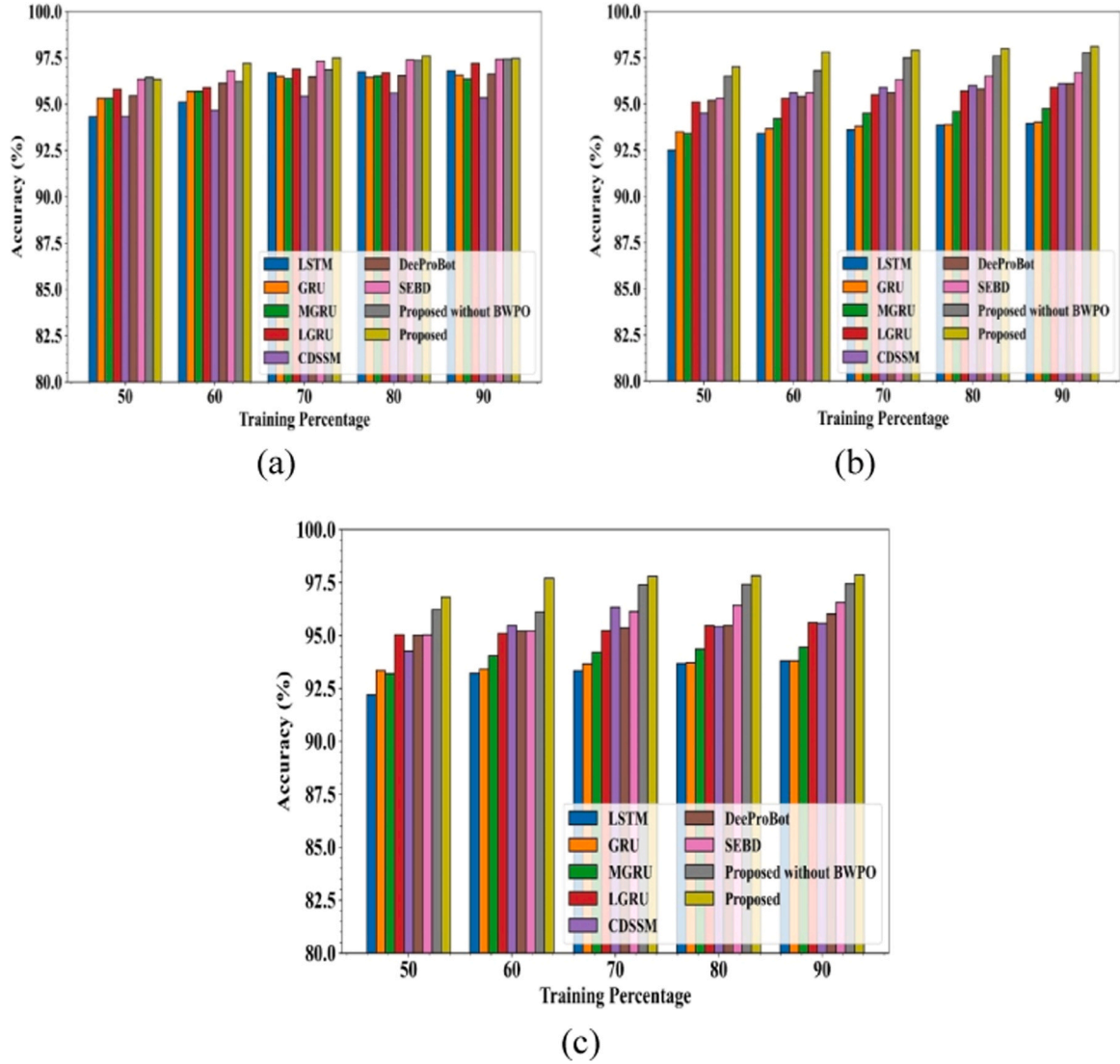


Fig. 15. Analysis based on data size variation: (a) Dataset 1 (Testset-1), (b) Dataset 1 (Testset-2), and (c) Dataset 3.

like behaviour patterns in tweets, allowing the proposed method to identify more spam bots.

Fig. 7 depicts a comparison of the proposed technique's F1 score for set 1 with known techniques such as LSTM, GRU, MGU, and LGRU. The graph clearly explains that the proposed technique achieves a better F1 score, with an F1 score of 99.2 %. The accuracy for the existing techniques is such that LSTM obtains an F1 score of 96.5 %, GRU achieves an F1 score of 96.2 %, MGU achieves a 96.2 % F1 score, and LGRU has an F1 score of 96.7 %. Thus, the existing techniques show poor F1 scores, thereby showing reduced performance. The proposed system's higher F1 score is due to the effectiveness of its classification technique in distinguishing between spam bots and real users. The integration of accurate feature representation, optimal feature subset selection using BWPO, and deep positional-aware classification results in a more accurate detection mechanism. It ensures that the model maintains an effective balance of precision and recall, resulting in a high F1-score for the proposed model.

Fig. 8 displays the suggested technique's error measures, notably MSE, RMSE, and MAE. The MSE of the suggested approach is 1.24, the RMSE is 1.114, and the MAE is 0.617 using Dataset 1 with Testset-1. The graph clearly demonstrates that the proposed technique has lower error values. The proposed model uses various feature extraction techniques to assist the model in capturing all relevant features, allowing the model

to extract finer elements that aid in enhancing spam bot behaviour. Then, the BWPO algorithm-based feature selection helps to reduce duplication, mitigate overfitting, and improve the model's generalization capabilities on previously unseen data. Finally, the PAtt_DenseCNN-based classifier with a positional attention mechanism enables the network to focus on the most informative segments of the input sequence. Then, accurate classification is made possible by employing dense convolutional layers to capture the local and global dependencies among the features. Thus, low MSE, RMSE, and MAE are acquired by the model in spam bot identification.

Fig. 9 depicts the processing time of the suggested technique when the input data size is varied. The processing time is less than 80 s when the input data size is 1500. As the size of incoming data increases, so does the processing time. The graph demonstrates that if the input data size is 6000, the processing time of the proposed technique exceeds 140 s.

Fig. 10 depicts the suggested technique's training and testing accuracy for various epochs. The accuracy starts increasing from epoch zero and reaches a peak at epoch 50. After that, the accuracy gradually increases up to the 150th epoch, and then it remains constant throughout the training and testing periods. In contrast to accuracy, the loss starts decreasing just after the 0th epoch and then shows a sudden decline. After that, it gradually decreases till the 150th epoch and then remains

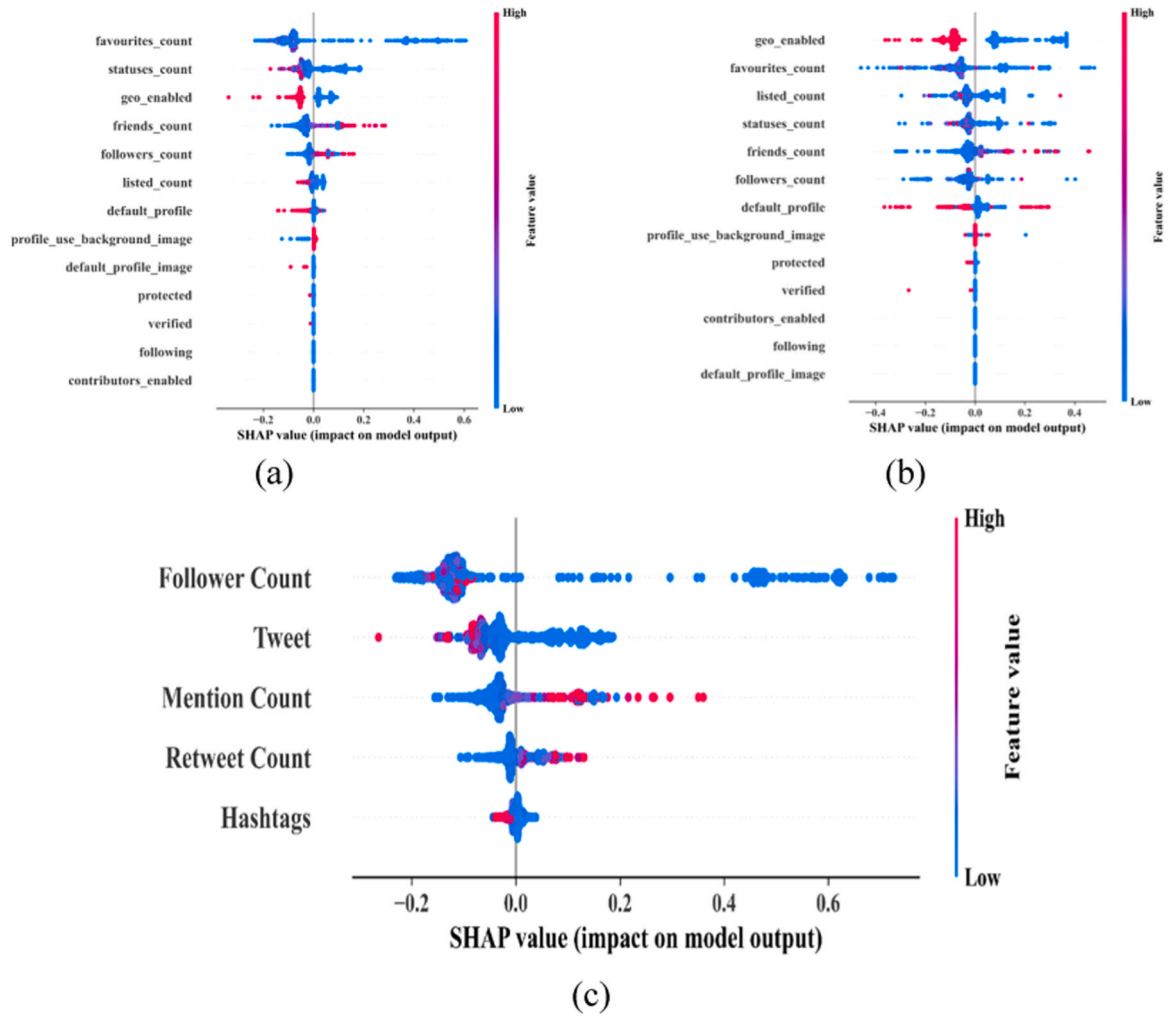


Fig. 16. Analysis based SHAP: (a) Dataset 1 (Testset-1), (b) Dataset 1 (Testset-2), and (c) Dataset 3.

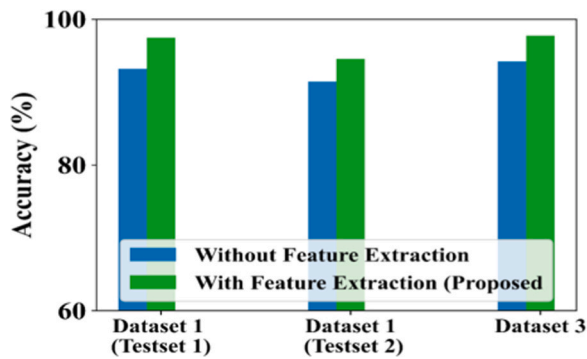


Fig. 17. Analysis of the proposed method with and without feature extraction.

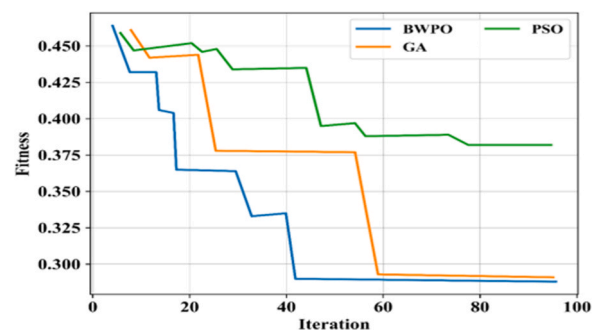


Fig. 18. Convergence analysis.

constant throughout the training and testing periods while evaluating Dataset-1 (Testset-1).

Fig. 11 displays the suggested technique's ROC curve in terms of true positive and false positive rates. The ROC curve is approaching one in this case, indicating that the suggested strategy correctly distinguishes between spam bots and actual users.

Fig. 12 illustrates the accuracy of the proposed technique by varying the batch size. If the batch size is 15, the accuracy is 93.31 %; when the batch size is 75, the accuracy is 94.3 %; if the batch size is 125, the accuracy of the proposed technique is 97.8 %; and if the batch size is 200, the accuracy is 96.5 %. The graph shows that when the batch sizes

vary, the accuracy also varies.

Fig. 13 depicts the suggested technique's accuracy for various learning rates. When the learning rate is 0.00001, the proposed technique's accuracy is 9.78 %; when the learning rate is 0.0005, the accuracy is 95.84 %. The accuracy is 94.95 % when the learning rate is 0.0001 and 93.39 % when the learning rate is 0.005 by Dataset-1 (Testset-1). The accuracy decreases with an increase in learning rate. The high learning rate causes the optimization process to overshoot the minima rather than gradually descend to the ideal parameter values. It makes the model acquire lower accuracy on the training with a higher learning rate.

Table 1
Comparison with existing methods.

Models	D1 Accuracy	D2	D3	D1 Precision	D2	D3	D1 Recall	D2	D3	D1 F1-Score	D2	D3
LSTM [29]	96.7	93.7	93.3	95.2	93.6	93.1	97.7	93.1	92.6	96.5	93.4	92.2
GRU [29]	96.5	93.8	93.7	94.8	93.5	93.4	97.7	93.4	92.7	96.2	93.4	91.4
MGRU [29]	96.4	93.7	94.2	94.9	93.3	94.2	97.5	93.4	93.4	96.2	93.3	93.2
LGRU [29]	96.9	93.9	95.2	95.6	93.8	95.1	97.7	93.4	94.3	96.7	93.6	95.5
C-DSSM [23]	95.4	92.5	96.3	95.1	93.2	96.5	97.0	91.2	95.5	98.1	92.2	96.2
DeeProBot [24]	96.5	93.3	95.4	96.2	94.1	95.3	96.1	92.2	96.4	97.3	93.1	95.3
SEBD [25]	97.3	92.5	96.1	96.2	92.3	96.2	97.1	93.3	95.3	98.2	91.3	97.4
Proposed without BWPO	96.9	93.1	97.4	96.1	94.0	97.2	97.3	93.9	96.0	96.3	92.2	96.2
Proposed	97.5	94.6	97.8	96.2	94.5	97.2	98.3	94.1	96.9	99.2	94.4	98.3

K-Fold based cross validation is demonstrated in Fig. 14 by considering the accuracy analysis. The proposed model shows enhanced accuracy with higher values of K in K-Fold cross-validation. A higher k-fold allows the model to be trained and validated on more diverse subsets of data, which reduces variance and improves model generalizability. Spam bot identification uses finer feature extraction patterns, and higher K-values ensure that complicated patterns are incorporated in at least some training and validation sets. Furthermore, the combination of PAtt_DenseCNN and BWPO-selected features improves the model's performance in higher k-fold variations.

Data-size variation based analysis is presented in Fig. 15. Here, the accuracy increases with larger dataset sizes. The proposed model with EBERT and vectorizer based significant feature extraction and PAtt_DenseCNN based spam detection with the attention mechanism assists the model in learning contextual features. It enhances the accuracy of detection in distinguishing spam bots. Also, the risk of overfitting and enhanced generalization is acquired by the proposed model with dense architectures guided by positional attention.

The Shapley Additive Explanations (SHAP) analysis of the proposed method is presented in Fig. 16. SHAP shows that TF-IDF, EBERT embeddings, and positional patterns are the most influential features in spam bot detection. SHAP analysis reveals that certain keyword or hashtag patterns, URL positions, and token sequence embeddings are highly indicative of spam. The model can identify suspicious patterns in the sequence with the aid of positional attention.

Analysis of the proposed method with and without feature extraction is presented in Fig. 17. Here, the feature extraction with the N-gram vectorizer assists in capturing short word patterns, and TF-IDF based feature extraction captures the words. Then, EBERT based feature extraction captures deep semantic context, and character-level vectorizer based feature extraction captures morphological patterns common in spam bots. Thus, spam bot detection with feature extraction assists the model in improving spam bot detection.

The convergence analysis of the BWPO algorithm employed for performing the feature selection along with the particle swarm optimization (PSO) and genetic algorithm (GA) algorithms is presented in Fig. 18. The PSO has an issue with premature convergence, whereas the GA has an issue with local optima trapping. However, the BWPO model balances exploration and exploitation using water wheel dynamics and plant resource distribution. As a result, the BWPO algorithm outperforms the GA and PSO algorithms in terms of convergence rate.

Discussion: The comparative analysis with existing methods of the best cases is presented in Table 1.

Here, the superior performance of the proposed spam bot detection model illustrates its efficiency. The pre-processing stage of the model enhances data quality by eliminating noise and extracting linguistic patterns. Furthermore, deep semantic information is extracted from tweet content using feature extraction techniques such as N-gram, TF-IDF, character-level vectorizer, and EBERT. The BWPO technique is used to choose the most pertinent feature subset that guarantees the selection of meaningful characteristics for learning in order to reduce the high dimensionality of the extracted features and prevent over-

fitting. In order to accurately classify spam bots from real users, PAtt_DenseCNN handles the final classification. It is effective at capturing spatial and contextual linkages within feature representations. Therefore, by combining many approaches, the suggested model achieves high accuracy, precision, recall, and F1-score while preserving low error rates such as MSE, RMSE, and MAE, processing data efficiently, and exhibiting excellent generalization, as indicated by a high AUC on the ROC curve.

4.3. Statistical analysis

The proposed study conducted statistical analysis to prove the efficacy of the proposed classification model. T-test and P-test are assessed and analyze the performance of the proposed PAtt_DenseCNN model.

P-test: The significance of the obtained classification results is proved by performing a P-test analysis. In this, the P-value is calculated to measure the evidence of results against the null hypothesis. The P-value should be less than 0.05, and the model that attains a reduced P-value is considered superior. In the proposed work, the P-value attained for dataset 1 is 0.035, dataset 2 is 0.041, and dataset 3 is 0.034. Thus, the results clearly prove the strength of the proposed detection model.

T-test: In this test, the means of two groups are compared together, and the possibility of two varied assumptions. Here, for the three datasets, a T-test is conducted and exhibits the ability of the proposed spam bot detection model. The null hypothesis is considered an initial assumption, where the true mean is assumed, and the obtained results are the same. The second assumption is a different hypothesis, in which the true mean and output results are presented as distinct. The proposed study conducts Welch's T-test, and the obtained P-value in Dataset 1 is 0.251, dataset 2 is 0.35, and dataset 3 is 0.243. Thus, the results show the ability of the proposed PAtt_DenseCNN model in spam bot detection.

4.4. Complexity analysis

Computational complexity is analyzed in this study to measure the performance of the proposed classification model and to highlight the inherent complications of computational issues. For evaluating the computational complexity of the proposed model in spam bot detection tasks, the proposed study has considered Big O notation. Because of utilizing an efficient architecture, the proposed study has attained reduced complexities of $O(1)$ for dataset 1, $O(nr^2 \log r)$ for dataset 2, and $O(1)$ for dataset 3. The analysis clearly exhibits that the proposed study has attained reduced complexity because of extracting and selecting the most optimal features using effective methods. On the other hand, the utilized position attention mechanism helps to focus the significant feature information and improves the learning ability of the proposed model. Similarly, the dense convolution operation learns the local feature information and reduces the feature dimensionality problems with the aid of pooling layers. Thus, by utilizing an effective method in each stage, the proposed work attained reduced complexities without impacting the overall performance.

5. Conclusion and future scope

PAtt_Dense CNN is a new deep learning-based classification framework developed in this study. The proposed technique efficiently classifies the tweets as spam bots or real users. Pre-processing techniques such as stop word removal, tokenization, stemming, n-gram identification, user mention, URL and hashtag removal, vocabulary density, and richness are used to remove irrelevant data and increase classification accuracy. The important characteristics are then retrieved using an N-gram level vectorizer, TF-IDF vectorizer, Character level vectorizer, and EBERT. The BWPO technique is then used to pick optimal features, hence lowering feature dimensionality. Next, spam bots and non-spam accounts are effectively classified utilizing the PAtt_Dense CNN framework. Finally, the performance of the suggested classification technique is evaluated using performance measures. LSTM, GRU, MGU, and LGRU are used to compare the results. The comparison results show that the proposed technique outperforms the existing techniques in terms of performance. As a result, the suggested technique produces more accurate classification results, with accuracy, precision, recall, and F1-Score of 97.8 %, 97.2 %, 98.3 %, and 99.2 %, respectively. Still, the accuracy of the model needs to be enhanced for processing real world applications. Thus, in the future, a lightweight deep learning model with loss optimization will be developed to improve the model's generalization potential by addressing the issues of over-fitting and computation burden. Additionally, information such as the user's geographic location and the time period of the tweets will be analyzed for spam detection.

CRediT authorship contribution statement

Hiren Joshi: Writing – review & editing, Visualization, Supervision, Resources, Methodology, Funding acquisition, Data curation. **Hemal Girishkumar Shah:** Writing – original draft, Validation, Software, Project administration, Investigation, Formal analysis, Conceptualization.

Funding Information

No funding is provided for the preparation of the manuscript.

Declaration of Competing Interest

No conflict of Interest.

Data availability

No data was used for the research described in the article.

References

- [1] L. Ilias, I. Roussaki, Detecting malicious activity in twitter using deep learning techniques, *Appl. Soft Comput.* 107 (2021) 107360.
- [2] S. Rao, A.K. Verma, T. Bhatia, A review on social spam detection challenges, open issues, and future directions, *Expert Syst. Appl.* 186 (2021) 115742.
- [3] A.P. Rodrigues, R. Fernandes, A. Shetty, K. Lakshmana, R.M. Shafi, Real-time twitter spam detection and sentiment analysis using machine learning and deep learning techniques, *Comput. Intell. Neurosci.* 2022 (2022).
- [4] N. Sun, G. Lin, J. Qiu, P. Rimba, Near real-time twitter spam detection with machine learning techniques, *Int. J. Comput. Appl.* 44 (4) (2022) 338–348.
- [5] Z. Alom, B. Carminati, E. Ferrari, A deep learning model for twitter spam detection, *Online Soc. Netw. Media* 18 (2020) 100079.
- [6] L. Luo, X. Zhang, X. Yang, W. Yang, Deepbot a deep neural network based approach for detecting twitter bots, in: *IOP Conference Series Materials Science and Engineering*, 719, IOP Publishing, 2020 012063.
- [7] M. Fazil, A.K. Sah, M. Abulaish, Deepbsd a deep neural network model with attention mechanism for socialbot detection, *IEEE Trans. Inf. Forensics Secur.* 16 (2021) 4211–4223.
- [8] C. Zhao, Y. Xin, X. Li, H. Zhu, Y. Yang, Y. Chen, An attention-based graph neural network for spam bot detection in social networks, *Appl. Sci.* 10 (22) (2020) 8160.
- [9] A. Derhab, R. Alawwad, K. Dehwah, N. Tariq, F.A. Khan, J.A. Muhtadi, Tweet-based bot detection using big data analytics, *IEEE Access* 9 (2021) 65988–66005.
- [10] M. Aljabri, R. Zagrouba, A. Shaahid, F. Alnasser, A. Saleh, D.M. Alomari, Machine learning-based social media bot detection a comprehensive literature review, *Soc. Netw. Anal. Min.* 13 (1) (2023) 20.
- [11] R.R. Rout, G. Lingam, D.V.L.N. Somayajulu, Detection of malicious social bots using learning automata with url features in twitter network, *IEEE Trans. Comput. Soc. Syst.* 7 (4) (2020) 1004–1018.
- [12] H. Shukla, N. Jagtap, B. Patil, Enhanced twitter bot detection using ensemble machine learning, 2021 6th International Conference on Inventive Computation Technologies (ICICT), IEEE, 2021, pp. 930–936.
- [13] P.G. Efthimion, S. Payne, N. Proferes, Supervised machine learning bot detection techniques to identify social twitter bots, *SMU Data Sci. Rev.* 1 (2) (2018) 5.
- [14] M. Heidari, J.H. Jones, O. Uzuner, Deep contextualized word embedding for text-based online user profiling to detect social bots on twitter, 2020 International Conference on Data Mining Workshops (ICDMW), IEEE, 2020, pp. 480–487.
- [15] A. Praveena, S. Smys, Effective spam bot detection using glow worm-based generalized regression neural network, *Mobile Computing and Sustainable Informatics Proceedings of ICMCSI 2021*, Springer, Singapore, 2022, pp. 469–487.
- [16] A.S. Abbas, Social network analysis using deep learning applications and schemes, *Soc. Netw. Anal. Min.* 11 (1) (2021) 106.
- [17] S. Khan, M. Fazil, V.K. Sejal, M.A. Alshara, R.M. Alotaibi, A. Kamal, A. R. Baig, BiCHATBiLSTM with deep CNN and hierarchical attention for hate speech detection, *J. King Saud. Univ. Comput. Inf. Sci.* 34 (7) (2022) 4335–4344.
- [18] K. Hayawi, S. Saha, M.M. Masud, S.S. Mathew, M. Kaosar, Social media bot detection with deep learning methods a systematic review, *Neural Comput. Appl.* 35 (12) (2023) 8903–8918.
- [19] M.D. Gutiérrez, G. Hernández-Peñaloza, A.B. Hernández, A.L. Díez, F. Álvarez, A deep learning approach for robust detection of bots in twitter using transformers, *IEEE Access* 9 (2021) 54591–54601.
- [20] B. Wu, L. Liu, Y. Yang, K. Zheng, X. Wang, Using improved conditional generative adversarial networks to detect social bots on twitter, *IEEE Access* 8 (2020) 36664–36680.
- [21] A.S. Alhassun, M.A. Rassam, A combined text-based and metadata-based deep-learning framework for the detection of spam accounts on the social media platform twitter, *Processes* 10 (3) (2022) 439.
- [22] N. Kausar, A. AliKhan, M. Sattar, Towards better representation learning using hybrid deep learning model for fake news detection, *Soc. Netw. Anal. Min.* 12 (1) (2022) 165.
- [23] R.R. Karwa, S.R. Gupta, Automated hybrid deep neural network model for fake news identification and classification in social networks, *J. Integr. Sci. Technol.* 10 (2) (2022) 110–119.
- [24] K. Hayawi, S. Mathew, N. Venugopal, M.M. Masud, P.H. Ho, DeeProBot a hybrid deep neural network model for social bot detection based on user profile data, *Soc. Netw. Anal. Min.* 12 (1) (2022) 43.
- [25] E. Alotthali, K. Hayawi, H. Alashwal, SEBD a stream evolving bot detection framework with application of PAC learning approach to maintain accuracy and confidence levels, *Appl. Sci.* 13 (7) (2023) 4443.
- [26] Z. Ellaky, F. Benabbou, Y. Matrane, S. Qaqa, A hybrid deep learning architecture for social media bots detection based on Bigru-LSTM and glove word embedding, *IEEE Access*, 2024.
- [27] A. Sallah, S. Agoujl, M.A. Wani, M. Hammad, A.A.A. El-Latif, Y. Maleh, Fine-tuned understanding: enhancing social bot detection with transformer-based classification, *IEEE Access* (2024).
- [28] G. Vyas, P. Vyas, P. Muzumdar, A. Chennamaneni, A. Rajavat, R. Rawat, Extracting and analyzing factors to identify the malicious conversational AI bots on twitter, *Conversat. Artif. Intell.* (2024) 71–83.
- [29] F. Wei, U.T. Nguyen, Twitter bot detection using neural networks and linguistic embeddings, *IEEE Open J. Comput. Soc.* (2023).
- [30] B. Jin, X. Xu, Wholesale price forecasts of Green grams using the neural network, *Asian J. Econ. Bank.* (2024).
- [31] B. Jin, X. Xu, Carbon emission allowance price forecasting for China guangdong carbon emission exchange via the neural network, *Glob. Financ. Rev.* 6 (1) (2024) 3491 (Applied Statistics).
- [32] B. Jin, X. Xu, Y. Zhang, Thermal coal futures trading volume predictions through the neural network, *J. Model. Manag.* (2024).
- [33] B. Jin, X. Xu, Machine learning predictions of regional steel price indices for east China, *Ironmak. Steelmak.* (2024), 03019233241254891.
- [34] B. Jin, X. Xu, Pre-owned housing price index forecasts using Gaussian process regressions, *J. Model. Manag.* 19 (6) (2024) 1927–1958.
- [35] B. Jin, X. Xu, Palladium price predictions via machine learning, *Mater. Circ. Econ.* 6 (1) (2024) 32.
- [36] B. Jin, X. Xu, Contemporaneous causality among price indices of ten major steel products, *Ironmak. Steelmak.* 51 (6) (2024) 515–526.
- [37] X. Xu, Contemporaneous causal orderings of US corn cash prices through directed acyclic graphs, *Empir. Econ.* 52 (2017) 731–758.